

Blatt 5

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: Passwort-Hashing

a) Anzahl möglicher MD5-Hashwerte

MD5 erzeugt einen 128-Bit-Hashwert. Damit gibt es $2^{128} \approx 3,4 \cdot 10^{38}$ mögliche Hashwerte.

b) Sicherheit des Verfahrens

Das Verfahren ist **nicht sicher**. Gelangt ein Angreifer an die Datenbank mit Benutzernamen und Passwort-Hashes, kann er:

- **Rainbow-Table-Angriff:** Vorberechnete Tabellen mit Hash-Passwort-Zuordnungen nutzen, um Passwörter in Sekunden nachzuschlagen.
- **Brute-Force-Angriff:** Alle möglichen Passwortkombinationen durchprobieren. MD5 ist sehr schnell berechenbar, was Brute-Force begünstigt.
- **Dictionary-Angriff:** Häufig verwendete Passwörter aus Wörterlisten hashen und vergleichen.
- **Gleiche Passwörter erkennen:** Nutzer mit identischem Passwort haben denselben Hash, was dem Angreifer sofort auffällt.

Das Passwort kann der Angreifer zwar nicht direkt aus dem Hash ableiten (Einwegfunktion), aber durch die genannten Methoden effizient ermitteln.

c) Verbesserungsvorschläge

- **Salt verwenden:** Jedem Passwort wird ein zufälliger Wert (Salt) hinzugefügt, bevor der Hash gebildet wird. Dadurch haben gleiche Passwörter unterschiedliche Hashes und Rainbow-Tables werden nutzlos.
- **Langsame Hashfunktion nutzen:** Statt MD5 spezielle Passwort-Hashfunktionen wie bcrypt, scrypt oder Argon2 verwenden. Diese sind bewusst rechenintensiv, um Brute-Force-Angriffe deutlich zu verlangsamen.

- **Pepper hinzufügen:** Zusätzlich zum Salt einen serverseitigen geheimen Wert (Pepper) verwenden, der nicht in der Datenbank gespeichert wird.

d) Unterschied zwischen Verschlüsselung und Hash

- **Verschlüsselung** ist **umkehrbar**: Mit dem passenden Schlüssel lässt sich der Klartext wiederherstellen. Ziel ist die Vertraulichkeit der Daten.
- **Hash** ist eine **Einwegfunktion**: Aus dem Hashwert kann der ursprüngliche Eingabewert nicht rekonstruiert werden. Ziel ist die Integritätsprüfung oder der sichere Vergleich von Werten (z. B. Passwörter).

Aufgabe 2: MD5-Passwort knacken

Lösungsansatz

Das Programm `MD5BruteForce.java` probiert systematisch alle Kombinationen aus Großbuchstaben (A–Z) mit steigender Länge durch. Für jede Kombination wird der MD5-Hash berechnet und mit dem Ziel-Hash verglichen.

Quellcode

```
1 import java.security.MessageDigest;
2 import java.security.NoSuchAlgorithmException;
3
4 public class MD5BruteForce {
5
6     private static final String TARGET_HASH = "2B85116183BF5D56056CAA0DDFD0DF86";
7
8     public static void main(String[] args) throws NoSuchAlgorithmException {
9         MessageDigest md5 = MessageDigest.getInstance("MD5");
10        long startTime = System.currentTimeMillis();
11
12        for (int length = 1; length <= 10; length++) {
13            System.out.println("Teste Länge " + length + "...");
14            String result = bruteForce(md5, new char[length], 0);
15            if (result != null) {
16                long duration = System.currentTimeMillis() - startTime;
17                System.out.println("Passwort gefunden: " + result);
18                System.out.println("Laufzeit: " + duration + " ms");
19                return;
20            }
21        }
22
23        System.out.println("Kein Passwort gefunden.");
24    }
25
26    private static String bruteForce(MessageDigest md5, char[] current, int pos) {
27        if (pos == current.length) {
28            String candidate = new String(current);
29            md5.update(candidate.getBytes());
30            byte[] hash = md5.digest();
31            String hexValue = byteArrayToHex(hash);
32            if (hexValue.equals(TARGET_HASH)) {
33                return candidate;
34            }
35            return null;
36        }
37        for (char c = 'A'; c <= 'Z'; c++) {
38            current[pos] = c;
39            String result = bruteForce(md5, current, pos + 1);
40            if (result != null) {
41                return result;
42            }
43        }
44        return null;
45    }
46
47    private static String byteArrayToHex(byte[] a) {
48        StringBuilder sb = new StringBuilder(a.length * 2);
49        for (byte b : a) {
50            sb.append(String.format("%02x", b));
51        }
52        return sb.toString();
53    }
54 }
```

```
36     }
37
38     for (char c = 'A'; c <= 'Z'; c++) {
39         current[pos] = c;
40         String result = bruteForce(md5, current, pos + 1);
41         if (result != null) {
42             return result;
43         }
44     }
45     return null;
46 }
47
48 public static String byteArrayToHex(byte[] a) {
49     StringBuilder sb = new StringBuilder(a.length * 2);
50     for (byte b : a)
51         sb.append(String.format("%02x", b));
52     return sb.toString().toUpperCase();
53 }
54 }
```

Code Listing 1: MD5BruteForce.java

Ergebnisse

a) Laufzeit

Das Programm lief durchschnittlich **9087 ms** (Durchschnitt aus 10 Durchläufen, AMD Ryzen 7 7840HX, 32 GB RAM).

b) Gesuchtes Passwort

Das Passwort lautet: **GRDIT**

c) Weitere Lösungen

Theoretisch sind Kollisionen möglich, da die Menge aller möglichen Eingaben unendlich ist, aber nur 2^{128} verschiedene Hashwerte existieren. Für kurze Passwörter aus Großbuchstaben ist eine Kollision jedoch extrem unwahrscheinlich. Im getesteten Bereich (Länge 1–5) gibt es keine weitere Lösung.

d) Beschleunigung der Suche

- **Multithreading:** Die Suche auf mehrere Kerne verteilen, z. B. jeder Thread bearbeitet einen Anfangsbuchstaben.

- **Rainbow Tables:** Vorberechnete Hash-Tabellen verwenden, um die Suche auf einen Lookup zu reduzieren.
- **GPU-Computing:** Grafikkarten können MD5-Hashes massiv parallel berechnen (z. B. mit Hashcat).
- **Dictionary-Angriff:** Häufige Passwörter zuerst testen, bevor alle Kombinationen durchprobiert werden.

Aufgabe 3: Diffie-Hellman Schlüsselaustausch

Gewählte Parameter

- Primzahl: $p = 23$
- Generator: $g = 5$
- Geheimer Schlüssel von Alice: $a = 4$
- Geheimer Schlüssel von Bob: $b = 3$

a) Rechenschritte

Schritt 1: Öffentliche Werte berechnen

Alice berechnet:

$$A = g^a \mod p = 5^4 \mod 23 = 625 \mod 23 = 4$$

Bob berechnet:

$$B = g^b \mod p = 5^3 \mod 23 = 125 \mod 23 = 10$$

Schritt 2: Öffentliche Werte austauschen

Alice sendet $A = 4$ an Bob. Bob sendet $B = 10$ an Alice.

Schritt 3: Gemeinsamen Schlüssel berechnen

Alice berechnet:

$$s = B^a \mod p = 10^4 \mod 23 = 10000 \mod 23 = 18$$

Bob berechnet:

$$s = A^b \mod p = 4^3 \mod 23 = 64 \mod 23 = 18$$

b) Gemeinsamer Sitzungsschlüssel

Beide Seiten berechnen denselben Schlüssel: $s = 18$

Quellcode

```

1 import java.math.BigInteger;
2
3 public class DiffieHellman {
4
5     public static void main(String[] args) {
6         BigInteger p = BigInteger.valueOf(23);
7         BigInteger g = BigInteger.valueOf(5);
8         BigInteger a = BigInteger.valueOf(4);
9         BigInteger b = BigInteger.valueOf(3);
10
11         System.out.println("=== Diffie-Hellman Schlsselaustausch ===");
12         System.out.println();
13         System.out.println(" ffentliche Parameter:");
14         System.out.println("  p = " + p + " (Primzahl)");
15         System.out.println("  g = " + g + " (Generator)");
16         System.out.println();
17
18         System.out.println("Schritt 1: Geheime Schl ssel w hlen");
19         System.out.println("  Alice w hlt a = " + a);
20         System.out.println("  Bob w hlt  b = " + b);
21         System.out.println();
22
23         BigInteger A = g.modPow(a, p);
24         System.out.println("Schritt 2: ffentliche Werte berechnen");
25         System.out.println("  Alice: A = g^a mod p = " + g + "^" + a + " mod " + p + " = " + A
26 );
27         BigInteger B = g.modPow(b, p);
28         System.out.println("  Bob:  B = g^b mod p = " + g + "^" + b + " mod " + p + " = " + B
29 );
30         System.out.println();
31         System.out.println("Schritt 3: ffentliche Werte austauschen");
32         System.out.println("  Alice sendet A = " + A + " an Bob");
33         System.out.println("  Bob sendet  B = " + B + " an Alice");
34         System.out.println();
35         BigInteger sAlice = B.modPow(a, p);
36         BigInteger sBob = A.modPow(b, p);
37         System.out.println("Schritt 4: Gemeinsamen Schl ssel berechnen");
38         System.out.println("  Alice: s = B^a mod p = " + B + "^" + a + " mod " + p + " = " +
39 sAlice);
40         System.out.println("  Bob:  s = A^b mod p = " + A + "^" + b + " mod " + p + " = " +
41 sBob);

```



```
40     System.out.println();
41
42     System.out.println("Ergebnis: Gemeinsamer Sitzungsschlüssel = " + sAlice);
43     System.out.println("Schlüssel stimmen überein : " + sAlice.equals(sBob));
44 }
45 }
```

Code Listing 2: DiffieHellman.java

Aufgabe 4: RSA-Verschlüsselung

a) Schlüsselerzeugung

Schritt 1: Zwei Primzahlen wählen

$$p = 5, \quad q = 11$$

Schritt 2: n berechnen

$$n = p \cdot q = 5 \cdot 11 = 55$$

Schritt 3: Eulersche Phi-Funktion berechnen

$$\varphi(n) = (p - 1) \cdot (q - 1) = 4 \cdot 10 = 40$$

Schritt 4: e wählen mit $\gcd(e, \varphi(n)) = 1$

$$e = 3, \quad \gcd(3, 40) = 1$$

Schritt 5: d berechnen mit $e \cdot d \equiv 1 \pmod{\varphi(n)}$

$$d = 27, \quad \text{da } 3 \cdot 27 = 81 = 2 \cdot 40 + 1 \equiv 1 \pmod{40}$$

Ergebnis:

- Öffentlicher Schlüssel: $(n, e) = (55, 3)$
- Privater Schlüssel: $(n, d) = (55, 27)$

b) Verschlüsselung

Die Nachricht $m = 7$ wird mit dem öffentlichen Schlüssel verschlüsselt:

$$c = m^e \pmod{n} = 7^3 \pmod{55} = 343 \pmod{55} = \mathbf{13}$$

c) Entschlüsselung

Der Chiffretext $c = 13$ wird mit dem privaten Schlüssel entschlüsselt:

$$m = c^d \mod n = 13^{27} \mod 55$$

Berechnung mittels Square-and-Multiply:

$$\begin{aligned} 13^1 &= 13 \\ 13^2 &= 169 \mod 55 = 4 \\ 13^4 &= 4^2 = 16 \\ 13^8 &= 16^2 = 256 \mod 55 = 36 \\ 13^{16} &= 36^2 = 1296 \mod 55 = 31 \\ 13^{27} &= 13^{16} \cdot 13^8 \cdot 13^2 \cdot 13^1 \\ &= 31 \cdot 36 \cdot 4 \cdot 13 \\ &= (31 \cdot 36) \cdot (4 \cdot 13) \\ &= (1116 \mod 55) \cdot (52 \mod 55) \\ &= 16 \cdot 52 \\ &= 832 \mod 55 = \mathbf{7} \end{aligned}$$

Die Entschlüsselung ergibt korrekt die ursprüngliche Nachricht $m = 7$.

Quellcode

```

1 import java.math.BigInteger;
2
3 public class RSA {
4
5     public static void main(String[] args) {
6         BigInteger p = BigInteger.valueOf(5);
7         BigInteger q = BigInteger.valueOf(11);
8         BigInteger message = BigInteger.valueOf(7);
9
10        System.out.println("=== RSA Schlsselerzeugung ===");
11        System.out.println();
12
13        System.out.println("Schritt 1: Zwei Primzahlen w hlen");
14        System.out.println("  p = " + p);
15        System.out.println("  q = " + q);
16        System.out.println();
17
18        BigInteger n = p.multiply(q);
19        System.out.println("Schritt 2: n = p * q = " + p + " * " + q + " = " + n);

```

```

20     System.out.println();
21
22     BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
23     System.out.println("Schritt 3:  (n) = (p-1) * (q-1) = " + p.subtract(BigInteger.ONE)
24         + " * " + q.subtract(BigInteger.ONE) + " = " + phi);
25     System.out.println();
26
27     BigInteger e = BigInteger.valueOf(3);
28     System.out.println("Schritt 4: e w hlen mit gcd(e,  (n)) = 1");
29     System.out.println(" e = " + e);
30     System.out.println(" gcd(" + e + ", " + phi + ") = " + e.gcd(phi));
31     System.out.println();
32
33     BigInteger d = e.modInverse(phi);
34     System.out.println("Schritt 5: d berechnen mit e * d  1 mod  (n)");
35     System.out.println(" d = e^(-1) mod  (n) = " + e + "^(-1) mod " + phi + " = " + d);
36     System.out.println(" Probe: e * d mod  (n) = " + e + " * " + d + " mod " + phi
37         + " = " + e.multiply(d).mod(phi));
38     System.out.println();
39
40     System.out.println(" ffentlicher Schl ssel: (n, e) = (" + n + ", " + e + ")");
41     System.out.println("Privater Schl ssel:  (n, d) = (" + n + ", " + d + ")");
42     System.out.println();
43
44     System.out.println("=== Verschl sselung ===");
45     BigInteger cipher = message.modPow(e, n);
46     System.out.println(" Nachricht m = " + message);
47     System.out.println(" c = m^e mod n = " + message + "^" + e + " mod " + n + " = " +
48     cipher);
49     System.out.println();
50
51     System.out.println("=== Entschl sselung ===");
52     BigInteger decrypted = cipher.modPow(d, n);
53     System.out.println(" m = c^d mod n = " + cipher + "^" + d + " mod " + n + " = " +
54     decrypted);
55     System.out.println();
56     System.out.println("Entschl sselung korrekt: " + message.equals(decrypted));
57 }

```

Code Listing 3: RSA.java